

CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU
CURSO DE CIÊNCIA DA COMPUTAÇÃO

DOCUMENTAÇÃO FRONT-END (REACT.JS)

Desenvolvimento da tela de editar perfil do site Not Fat

Responsável

João Carlos Oliveira da Silva - 01806272

Disciplina: Códigos de Alta Performance - Web
Professor(a): Anderlan Henrique Batista Siqueira

Maceió - AL
2026

Resumo

Essa tela é o Painel de Edição de Perfil do usuário. Quando ela é aberta, faz uma conexão com o servidor para buscar os dados mais recentes (nome e e-mail). Na interface, o usuário pode alterar o seu nome completo, mas o e-mail é exibido de forma bloqueada (apenas leitura) por questões de regra de negócio ou segurança. Ao finalizar as edições e clicar em "Salvar", a tela envia o novo nome para o banco de dados, atualiza as informações locais (no navegador e no contexto da aplicação) e redireciona o usuário de volta para a sua página de perfil.

1 Funções da tela de editar perfil

1.1 Estados e Ferramentas (Linhas 7 a 13)

O componente começa preparando tudo o que precisa para funcionar:

`useState`: Cria variáveis de estado para guardar o que está sendo digitado.

`nome`: Guarda o valor em tempo real enquanto o usuário digita no campo de texto.

`nomeSalvo`: Guarda o nome confirmado para ser exibido no "cartão" de apresentação logo acima do formulário.

`email`: Guarda o e-mail do usuário (que não será alterado).

`useNavigate`: Ferramenta do React Router para redirecionar o usuário para outra página (como o botão "Cancelar" ou após salvar com sucesso).

`useAuth`: Traz a função `updateUser`, que serve para avisar ao resto do aplicativo que o nome do usuário mudou, sem precisar recarregar o site.

1.2 Carregamento dos Dados com `useEffect` (Linhas 15 a 52)

Este bloco roda automaticamente apenas uma vez, assim que a tela é aberta.

Ele pega os dados temporários do usuário no `localStorage` (armazenamento do navegador).

Em seguida, faz uma requisição (`fetch`) para o seu servidor backend (`http://localhost:3000/usuario`), pedindo os dados mais atualizados do banco de dados.

Se a API responder com sucesso (`response.ok`): Ele preenche os estados (`nome`, `email`, etc.) com os dados do banco e já aproveita para atualizar o `localStorage` para manter tudo sincronizado.

Se a API falhar (cair no `catch`): Ele usa os dados antigos que já estavam no `localStorage` como um plano B, garantindo que a tela não fique em branco.

1.3 Função de Salvamento `handleSave` (Linhas 54 a 83)

Esta função é acionada quando o usuário clica em "Salvar alterações".

Ela monta uma requisição do tipo `PUT` (usada para atualizações) e envia o novo `_nome` em formato `JSON` para o servidor.

Se o servidor aceitar a mudança (`response.ok`):

Cria um objeto `usuarioAtualizado` com o novo nome.

Atualiza o estado global da aplicação (`updateUser`).

Atualiza o armazenamento do navegador (`localStorage`).

Redireciona o usuário de volta para a tela `/profile` usando o `Maps`.

Se der erro: Exibe um `alert` informando ao usuário que não foi possível salvar ou conectar ao servidor.

1.4 Estrutura Visual (Interface JSX)

A partir da linha 84 (`return`), temos o desenho da interface usando Tailwind CSS. A tela é dividida estrategicamente:

Cabeçalho e Botão Voltar: Um link no topo com o ícone `ChevronLeft` que permite voltar ao perfil rapidamente.

Cartão de Resumo do Usuário: Uma área que mostra um avatar verde e os dados atuais do usuário (`nomeSalvo` e `email`). Aqui, há um aviso visual com um cadeado (`<Lock />`) avisando que o e-mail é fixo.

Campos de Formulário (Inputs):

Nome: O código traz uma melhoria de acessibilidade ligando a etiqueta (`<label htmlFor="input-nome»`) diretamente ao campo de digitação (`<input id="input-nome»`). O valor desse campo é atrelado ao estado `nome` e atualizado com o `onChange`.

E-mail: Um campo de `input` desativado (`disabled`), com opacidade reduzida e cursor de "não permitido" (`cursor-not-allowed`) para reforçar que não é editável.

Botões de Ação: O rodapé do formulário contém o botão de Cancelar (que usa `Maps` para voltar) e o botão de Salvar alterações, que chama a função `handleSave` explicada no item 3.